采集层设计：棘轮页的数据从哪来

当前版 · 2026-09-30 · 与《棘轮页方案.pdf》（第三份评审稿）配套 · 分支 `task/4db75afb` head `175dfafd`，最后一次采集 `aaed8e4f`

这份文件是什么

- 页面长什么样、每个数是多少，看同目录的《棘轮页方案.pdf》（第三份评审稿）。房间和 PR 的当前评审材料只有那一份。
- 本文件只讲采集层：快照里有什么、规则指纹怎么算、在哪跑、后端怎么存、历史怎么补、页面据什么判方向。
- 采集层已经实现并在真实树上跑通：10 道检查各自输出 `--json`，汇总脚本合成 `ratchet-snapshot.json`，`arch-metrics.yml` 在合入 main 后采集并上传。待办是后端存库与真实后台页面。

上一版的页面部分（第二版预览与采集设计：当时的截图、138 个场景的旧口径数字）已被这一稿替代。历史留在提交 `74aab1be`，本文件不再保留那些内容，免得两版数字并列被误读。

1　采集设计

1.1 采什么：每次一份快照

main 每次合入和每周一，CI 跑一遍全部检查，写出一份 `ratchet-snapshot.json`：

字段	含义
`commit`、`commit_date`	被检查的提交
`collected_at`、`run_url`	采集起止时间和 CI 运行链接
`checks[].id`、`area`	检查的稳定编号和所属方面（场景、边界、规模、类型与样式）
`checks[].evidence`	只用于场景：哪些场景有挂载证据（`catalog.spec.ts` 的逐格结果），和评分结果分开存
`checks[].status`	`pass` / `fail` / `cannot_judge` / `not_collected`，直接取检查器的退出码 0 / 1 / 2；没跑到的记 `not_collected`，不记 0
`checks[].actual`	检查器本次实际数出的违规数；检查器不输出计数时为 `null`
`checks[].frozen`	基线里登记的豁免数
`checks[].stale`	已经失效、还没清掉的豁免（逐条列出）
`checks[].better`	哪个方向算变好。取自检查器自己的声明和「架构指标」文档，不在页面里另定
`checks[].rule_fingerprint`	判定脚本和规则配置的哈希，见 1.2
`checks[].details`	按文件的明细，用于展开定位；每道最多 200 行
`board`	`arch-metrics.py` 的原样输出：热点、函数内导入等参考指标

每个检查器加一个只读的 `--json` 参数输出自己那一段，汇总脚本只负责拼接，不重算、不判定。如果 CI-fast 也要用这些 JSON：输出格式由本话题维护，CI-fast 只读；两边有冲突时以芝士 K 的 CI-fast 设计为准。场景和分级仍由 `frontend_grade.py` 判，汇总脚本只读 `scene-ratchet.py` 的输出。

1.2 规则指纹：把「口径变了」和「代码变了」分开

每道检查登记一组「规则文件」：判定脚本和规则配置，不含基线。指纹是这组文件内容的 SHA-256。

- 例：前端边界 = `eslint.boundary.config.mjs` + `import-boundary-ratchet*.mjs`；场景 = `frontend_grade.py` + `scene-ratchet.py`。
- 清单按实际调用图取，不手写：从该检查的入口出发，跟一遍它 import、读配置、读脚本的路径，读到的文件都算。所以共用核心（`tsc-ratchet-core.mjs`、`ratchet-report.mjs`）和它读的配置（`eslint.config.mjs`、`check_boundaries.py`）都在里面。手写清单会漏掉这类间接依赖，改动它们时指纹不动，规则变化就被算成还债。
- 两类文件不进指纹：基线只决定冻结数，改它不该开新段；检查对象（`CLAUDE.md`、`SKILL.md` 这类被读来判断的文件）改了就是测量结果本身，不是口径变了。
- `.importlinter` 里规则和少数例外写在同一个文件，指纹只算去掉 `ignore_imports` 之后的契约定义。2026-10-14 起这个文件不再装基线：基准改成每次检查从 `git merge-base HEAD origin/main` 那棵树量出来，文件里那几行是有人明确接受的例外，不是债。

- 指纹变了，页面从这一次起开新的比较段，并在走势上标「判定变更」，附那次提交和 PR。之前的点照样显示，但不参与「起点 → 现在」的差值。
- 取舍：只改了注释也会被当成判定变更，页面会多标一次。宁可多标让人看一眼，也不漏标把规则变化算成进步。真实例子：#2178 改写了 `check-file-sizes.py`，但上限没变。在 #2153 的提交上单独重跑看板，超出行数已经是 28,397，和 #2178 之后一样，所以这一步的下降全部来自 #2153。这种情况页面会标一次「判定变更」，由人看过后确认。

1.3 在哪跑

做法	怎么做	好处	代价
甲　扩展 arch-metrics（已实现）	在 `arch-metrics.yml` 的 job 里装前后端依赖，跑全部检查器，上传快照	不碰必需检查；全部检查同一个提交、同一次采集	工作机上实测 275 秒跑完 10 道检查，另加装依赖的时间；job 超时 45 分钟
乙　必需检查顺手存	`frontend.yml`、`test.yml` 在 main 上把结果写成 JSON 上传，arch-metrics 再汇总	几乎不多花时间	要改必需检查，是 CI-fast 的范围；几份结果来自不同 run，时间对不齐

=采集失败不能读成通过。某道检查的工具链不在，是那一道记 `not_collected`、`actual` 为 `null`；采集整体跑不成，是这一步红、退出码带出去，并留一份 `ratchet-snapshot.failed.txt` 说明原因，上传用 `if-no-files-found: error` 而不是只警告。CI 的 `run:` 体是 `bash -e`，退出码要用 `||` 接住再往下写证据——写在下一行的话，失败时那几行根本不会执行。回归在 `deploy/tests/test_arch_metrics_workflow.py`：它把这一步和摘要步自己的正文取出来，用 `bash -e` 跑。

1.4 后端怎么拿、存在哪

- 后端复用已有的 GitHub Actions 读取代码（`review/github_pr.py` 里已经在调用 runs 接口），用平台 App 的读令牌列出名为 `ratchet-snapshot` 的工件，下载、校验版本号后存库。
- 新表存一行一份快照：提交、run 编号（唯一）、采集时间、快照版本、JSON 正文。存库后不受 CI 工件 90 天保留期的限制。
- 后台定时任务每 15 分钟拉一次；页面上的「刷新」按钮立即拉一次。页面只读库，打开页面不直接请求 GitHub。
- 页面同时显示「采集提交」和「当前部署提交」。两者不同时照实写两个 SHA，不假装同步。

1.5 历史补录

已有的 55 份 `arch-metrics` 记录只存了冻结数，没有实际数，也没有指纹。一次性补录脚本在 CI 里带完整历史运行，逐个提交回算各检查的规则指纹，把旧记录按指纹分段导入。回算不了的标「口径未记录」，只画点，不参与差值。

1.6 页面上怎么判「在降 / 没动 / 新增」

相邻两次采集之间可能隔着好几个提交：CI 会取消被新推送顶掉的运行。所以一步变化挂的是这一段区间里的全部 PR，不记在区间末尾那一个上。实例：`90f83dd..95e6aeb` 之间有 #2153 和 #2178 两个提交，超出行数的下降来自 #2153 删除后端接口，不是 #2178 改 CI。

只比同一指纹段内的首尾两点，方向按 `better` 字段。页面不打分、不加权、不把几道检查合成一个总分。「新增豁免」只在冻结数上升且指纹没变时标出，并列出那次提交。

2　外部做法

以下来源都在 2026-09-30 查阅原文。「实证」是原文写的，「推断」是我的判断。

来源	机制（实证）	对本设计
SonarQube 2026.2 Activity 页 docs.sonarsource.com/sonarqube-server/2026.2/user-guide/viewing-projects/activity-and-history	历史图上有独立的事件类型：`Profile`（规则集被改或换了，附规则变更明细）、`Definition change`、`Issue detection`（分析器升级）。事件和度量值分开记。	**采用**。「判定变更」做成独立事件，挂在走势上，不改数值本身。这就是 1.2 的规则指纹
import-linter 2.15 文档 import-linter.readthedocs.io/en/stable/contract_types/	每条契约可设 `unmatched_ignore_imports_alerting` = error / warn / none，豁免匹配不到时报警。	**采用**报警语义，本仓库已设为 warn。**不采用**它的输出当数据源：由 `check_boundaries.py` 解析后输出 JSON

	命令行只有文本输出，退出码 0 / 1，没有 JSON。	
ESLint 批量豁免（v9.24 引入） eslint.org/docs/latest/use/suppressions	`eslint-suppressions.json` 按「文件 + 规则」记计数；实际数超过冻结数时报出全部违规；有失效的豁免时非零退出并提示 `--prune-suppressions`。	**部分采用**：计数粒度和「失效豁免要显式清」的做法与本仓库基线一致。不迁移到它：只管 error 级、无逐条身份，现有基线脚本已覆盖
betterer 5.4.0 github.com/phenomnomnominal/betterer	结果文件按「文件哈希 + 每条问题哈希」存；状态有 better / worse / same / new 和到期（expired）。测试定义改名时旧条目被当成缺失，下次写入时静默丢掉。	**不采用**作实现：它识别不了规则变更，正是要防的情况。借鉴逐条哈希的思路，留给以后逐条追踪豁免
GitHub Actions 工件接口 docs.github.com/en/rest/actions/artifacts	列表支持按 `name` 过滤；带 `workflow_run.head_sha`、`created_at`、`expired`；下载是 302，链接 1 分钟失效；需要 Actions 读权限；公开库最多保留 90 天。	**采用**作传输：字段够标提交和采集时间。**不当历史库**：会过期，所以后端存库
Thoughtworks「Fitness function-driven development」2019 thoughtworks.com/insights/articles/fitness-function-driven-development	把架构目标写成能自动检查的函数，用来客观衡量技术债。原文没讲基线和棘轮。	只借定位：棘轮页展示的是这些检查的结果，不自己打分

查过但不作参考：Multica（github.com/multica-ai/multica）和 Buzz（github.com/block/buzz）都没有棘轮或边界检查的报表，Buzz 的 metrics 是运行时 Prometheus 指标。关于「Shopify / GitHub 用棘轮还 lint 债」的博客，没找到一手来源，不引用。

3　落地状态与顺序

步骤	状态
10 道检查的 `--json` 输出	已实现。自测：同一棵树跑两种模式退出码一致、`--json` 只打一行、判不了记 `cannot_judge` 并带原因，不记 0
汇总脚本 `ratchet-snapshot.py` 与 `arch-metrics.yml` 的采集上传	已实现（head `175dfafd`）。采集用后端 venv 或系统 python 起，不依赖 `uv run`；失败留证据、不吞退出码，见 1.3 末段。回归 `deploy/tests/test_arch_metrics_workflow.py`
后端表和定时拉取	未做，采集这一步合入 main 之后开
真实后台页 `AdminRatchetPage.vue`	未做。接口就绪后开，另出一份可照着实现的界面 PDF 给评审

采集这一步仍是本话题的一个 PR，页面结构用户定了之后才动界面。